

ADR-009: Preview clips, cut without a decoder

Status: Accepted 2026-11-03. Not yet implemented.

Context: Jay, playing the fork with the server library browsing: *"the song preview does not load"*. Correct, and by design — `RemoteSongEntry.LoadPreviewAudio` returns `null`, because a song the server published and this disk does not hold has no bytes to preview. Jay asked for a server endpoint instead of a client-side download.

What the server already has

Nothing needs adding to the catalogue. Every song already carries:

Field	Source
`preview_start_ms`, `preview_end_ms`	`internal/scan/scan.go:229`, from `song.ini`
`assets.stems`	`internal/scan/scan.go:422`
stem classification	`internal/scan/names.go:87` `ClassifyStem`

And `internal/sng` implements `fs.FS`, so a stem inside a package opens as an ordinary file.

The decision

Cut the clip at container boundaries. Do not decode, do not re-encode, add no dependency.

Why not ffmpeg

This was the obvious first answer and it is wrong here, for a reason that is specific to this deployment rather than general. The runtime image is `gcr.io/distroless/static-debian12:nonroot` (`Dockerfile:19`) — **static distroless: no shell, no package manager, nothing to install into.** Adding ffmpeg means replacing the base image and carrying roughly 60–100 MB of it, against ADR-001's "tiny container" and the Raspberry Pi target that is a project promise rather than a bonus. ADR-003 already set the precedent for pricing a dependency before taking it: `sevenzip` cost 3 → 71 packages and +1.62 MB, and that was judged worth it. An ffmpeg layer is two orders of magnitude more and would also need an arm64 build for the Pi.

Why not range-serve the stem

The cheap alternative — serve bytes *N*..*M* of the stem and let the client's BASS decoder resync — was offered and then **ruled out by measurement**, not by taste. Counted across the live catalogue, 22,091 stem files:

```
.ogg     20288
.opus     1354
.mp3       449
```

98% is Ogg-family. An Ogg file declares its codec, sample rate and channel count in header pages at the *start* of the file; hand a decoder bytes from the middle and there is nothing to decode with. Range-serving would have worked for the 449 MP3 stems and failed for the other 21,642. It is not a cheaper option, it is a broken one.

What "cut without decoding" means

An `.ogg` file is not one blob. It is a sequence of self-describing **pages**, each 27 bytes of header plus a segment table plus payload, carrying a `granule_position` that says how far into the stream the page ends. Playing audio means turning pages into sound, which needs a codec. *Cutting* audio means copying whole pages, which needs only the ability to find where pages begin and end — reading headers, not decoding them.

The same shape holds for the other two formats, which is why one approach covers all of them:

Format	Unit	How its position is known
Ogg Vorbis	page	`granule_position`, in samples at the rate in the identification header
Ogg Opus	page	`granule_position`, always in 48 kHz units regardless of input rate
MP3	frame	fixed duration per frame, found by the sync word in each frame header
WAV	byte	raw samples; pure arithmetic

So the algorithm, for the 98% case:

1. copy the leading pages whose `granule_position` is 0 — those are the headers (`\x01vorbis` identification plus comment and setup, or `OpusHead` plus `OpusTags`), and copying them is what makes the output a standalone playable file rather than a fragment
2. read the sample rate out of the identification packet, so granule positions convert to time
3. skip pages until `granule_position >= preview_start`
4. copy pages until the clip is long enough
5. set the EOS flag on the last page, renumber the page sequence, and **recompute the CRC** of any page whose header was touched

Output: a valid Ogg of a few hundred KB instead of a ~50 MB download.

What this costs, stated honestly

It is a parser, and parsers are where this project has found most of its defects. Two known wrinkles rather than surprises:

- **Vorbis uses overlapping windows**, so the first audio page of a cut clip has no predecessor to overlap with and the first ~20 ms can glitch. Acceptable for a preview; worth knowing before somebody reports it as corruption.
- **Granule positions must be rebased** so the clip starts near zero, or a player reads it as a seek into a longer stream and may report a wrong duration.

How it must be verified, and what would not count

A clip this code calls valid is not a clip somebody can play. The repo already has this rule in two forms: "*a forgiving reader hides a strict writer, so exercise a new container by EXTRACTING it*", and "*round-tripping against our own reader only proves our reader and writer agree with each other.*" Both apply exactly. A cutter and a parser written in the same afternoon will agree with each other about a malformed page.

So the standard is an **independent decoder**: the clip must play in the real YARG client, which is the oracle this project already uses for packaging questions, and `BASS` is not our code. A Go test asserting page structure is necessary and is not sufficient, and must not be reported as proof the feature works.

The unit tests build Ogg streams by hand — pages with known granule positions — because that tests the page arithmetic precisely and needs no fixture. Per the repo's rule, the cut must be seen to go **wrong** first: a test that cuts at a known boundary and asserts the wrong page count should fail before the correct one is trusted.

The endpoint

```
GET /song/{chart_hash}/preview.ogg
```

- 404 when the hash is unknown, as `/song/{hash}.sng` already does
- 415 when the stem is a format this cannot cut
- strong `ETag`, because the output is deterministic — the same package and the same `preview_start_ms` always yield the same bytes, which is the property ADR-002 and the packer's deterministic mask already protect
- cached on disk beside the packed archives, so a clip is cut once

Which stem. `ClassifyStem` can report several. The preview should use the full mix where a song has one, and otherwise the loudest-weighted guitar or vocals stem rather than an arbitrary first entry — mixing stems would mean decoding, which is the one thing this design avoids. That choice is a correctness question for the oracle, not a guess to be made here.

Not in scope

- **Mixing multiple stems.** Needs a decoder. A single stem is a worse preview than YARG's own and is still a preview.
- **Transcoding to one format.** Clients get the container the stem was in. YARG decodes all three already.
- **Generating previews at ingest.** On-demand plus a cache gets the same result without a migration or a rescan of 4,085 songs.